9 Kotlin

This chapter covers

- Why use Kotlin?
- Convenience and conciseness
- Safety
- Concurrency
- Java interoperability

Kotlin is a language created by JetBrains (**https://jetbrains.com**), makers of the popular IntelliJ IDEA. Announced publicly in 2011, Kotlin aims to fill language gaps they felt developing in Java, without the friction they saw with other existing JVM languages.

Kotlin was open sourced the following year and reached 1.0—with guaranteed levels of support and maintenance from JetBrains—in 2016. Since then, it has gone on to become the recommended language for the Android platform and has gathered a solid following in other JVM coding circles. The Kotlin Foundation, announced in 2018, brings the support of both JetBrains and Google to the language for the long term. Kotlin is even reaching beyond the JVM to support JavaScript and native backends.

9.1 Why use Kotlin?

As alternate languages go, Kotlin provides many quality of life improvements over Java, while not radically changing the entire world. Its focus on convenience, safety, and solid interoperability has given it a great story for incremental usage in existing Java projects. With first-class support in IntelliJ IDEA, turning a file from Java to Kotlin is often just a click away.

It's worth noting that some features originally available only in Kotlin have made their way back into Java. A great example of this is Kotlin script—Kotlin can run a source file directly, typically with the `kts` extension, without the developer asking to compile it. If this sounds like Java 11's single-file feature that we showed off in chapter 1, you're not wrong!

Following this chapter, we will use our new familiarity with Kotlin in chapter 11 as the main scripting language for our builds with Gradle. It will also be revisited in chapter 15 where we examine functional programming on the JVM, and in chapter 16 where Kotlin's built-in coroutine mechanism provides a compelling alternative to classical multithreading in Java. Let's start seeing what Kotlin can provide for us.

1.1 Installing

If you use IntelliJ IDEA, Kotlin is provided via a plugin. With that installed, you can start writing code in Kotlin straight away, just like any other supported language in the IDE.

For those who are more inclined to bare-bones setups, Kotlin also provides a command-line compiler (`kotlinc`) (see **http://mng.bz/YGoa**) and interactive shell (`kotlin`).

Adding Kotlin to an existing project requires updating your build scripts. Chapter 11 will get you more familiar with these systems, but for now, you can refer to Kotlin's excellent documentation to get you started with either Maven (**http://mng.bz/GEYJ**) or Gradle (**http://mng.bz/z4vA**). With Kotlin ready to run, a good place to start exploring is in how its basic features work and improve on Java.

9.2 Convenience and conciseness

Java has a reputation for being verbose. Although it retains a lot of visual similarity with Java, Kotlin relentlessly streamlines the code you have to write.

2.1 Starting with less

One example of its streamlined nature is the simple character, the semicolon. Kotlin doesn't require semicolons for line ending, allowing the typical newlines to stand in their place. Semicolons are allowed—and, in fact, required if you want to put multiple statements on a single line, for instance—but most often, they aren't needed.

Kotlin took advantage of its blank slate to alter other defaults that would be difficult to change in Java. Although Java imports only `java.lang` by default, Kotlin provides the following packages everywhere:

- `java.lang.*`
- `kotlin.*`
- `kotlin.annotation.*`
- `kotlin.collections.*`
- `kotlin.comparisons.*`

- `kotlin.io.*`
- `kotlin.ranges.*`
- `kotlin.sequences.*`
- `kotlin.text.*`
- `kotlin.jvm.*`

It's the rare program that doesn't need Collections, text, or IO, so these inclusions save a lot of needless importing.

A spot where Java's verbosity often shows up is with types—and we saw in chapter 1 how the `var` keyword cuts down on repetition of type information. Kotlin has had this style of type inference since the beginning, although it is far from the only source for the idea.

2.2 Variables

When introducing a variable, Kotlin even uses the same keyword as newer Java releases: `var`. It will infer from the expression on the right-hand side what type to use for the variable, as shown here:

```
var i = 1          ❶
var s = "String"   ❷
```

❶ i is of type kotlin.Int.

❷ s is of type kotlin.String.

Unlike Java, though, in Kotlin `var` is not just a shortcut. If you want to explicitly call out the type of the variable, `var` remains, but the type is added after the variable name, as shown in the next code snippet. This is sometimes referred to as a *type hint* :

```
var i: Int = 1
var s: String = "String"
var n: Int = "error"       ❶
```

❶ This assignment will fail with a compile error, error: type mismatch: inferred type is String but Int was expected.

In Kotlin, `var` has a friend in the `val` keyword. Variables declared with `val` are immutable and cannot be written after assignment. It's the equivalent of `final var` in Java —a highly recommended default for any variable that isn't expected to be reassigned.

Kotlin nicely makes this safer setting just as concise as the mutable alternative, as illustrated here:

```
var i = 1
i = 2           ❶

val s = "String"
s = "boom"      ❷
```

❶ Reassignment is allowed to var variables.

❷ Compile error, error: val cannot be reassigned.

`var` and `val` are used consistently through Kotlin for variables and arguments. The theme of favoring immutability by default is also a key design factor in Kotlin that we'll see time and again throughout the language.

Once we have variables it's natural to want to compare them. Kotlin has some fresh help to give us on equality that's worth knowing about.

2.3 Equality

A common mistake in many Java programs is shown in the following code:

```
// Java
String s = retrieveString();  ❶
if (s == "A value") {
   // ...                     ❷
}
```

❶ Receives a string from somewhere. Note that string literals may actually be interned to the same object, giving a false sense of security in the incorrect comparison.

❷ Not reached, even when same value but separate references are used

We quickly learn that `==` in Java compares *references*, not *values*, so this doesn't do what you'd expect from many other languages.

Kotlin eliminates this quirk and treats `==` as value equality on common types such as `String`. Effectively, calls to `==` are equivalent to a `null`-safe call to `equals`. This opti-

mizes for the more common programming case and avoids a huge cause of errors in Java programming, as shown next:

```
// Kotlin
var s: String = retrieveString()
if (s == "A value") {
   // ...                    ❶
}
```

❶ Will be reached if the value of s is “A value”

On rare occasions, you may still have reason to compare references. When those cases arise, Kotlin’s `===` (and its pair `!==`) gives you the behavior Java used for `==` and `!=`. Comparing variables is important, but you won’t get far without needing to call other code or define your own subroutines.

2.4 Functions

Sometimes when speaking, we use the terms “function” and “method” interchangeably, but, in fact, Java only has methods—you can’t define a reusable block of code outside of the context of a class. Although Kotlin supports all the object-oriented goodness of Java—we’ll see it in an upcoming section—it also recognizes that sometimes you just want a function by itself.

Keeping to Kotlin’s principle of conciseness, a minimal function definition looks like this:

```
fun doTheThing() {    ❶
  println("Done!")
}
```

❶ fun defines a function in Kotlin. We’ll see it again when we get to defining methods attached to classes.

This looks a little different than the Java way of doing things, but it is still fairly recognizable. Apart from the `fun` keyword to start the declaration, the biggest difference is the lack of return type. In Kotlin if your function doesn’t return anything, rather than stating `void` explicitly, you can say nothing, and the return type is treated as `Unit` (Kotlin’s way of saying “no return value”).

If we do want to return a value, we must state that directly, as follows:

```
fun doTheThing(): Boolean {   ❶
  println("Done!")
  return true                 ❷
}
```

❶ Declares that our function returns type Boolean

❷ Returns our success

Functions aren't much use without taking arguments. Kotlin's syntax for that looks much like variable declaration, shown next:

```
fun doTheThing(value: Int): Boolean {   ❶
  println("Done $value!")               ❷
  return true
}
```

❶ Our function now takes a single argument of type Int.

❷ Arguments appear within the function like locally defined variables. Here we use one in Kotlin's handy string interpolation.

Even the simple function definition in Kotlin has a few tricks up its sleeve, though. Sometimes the order of arguments to a function can be unclear, particularly when types match between arguments, as in the following example:

```
fun coordinates(x: Int, y: Int) {
  // ...
}
```

When we call this function, we have to remember the order of the arguments—`x` comes before `y`—or risk a bug. Kotlin solves this with *named arguments*, shown next:

```
fun coordinates(x: Int, y: Int) {
  // ...
}

coordinates(10, 20)          ❶
coordinates(y = 20, x = 10)  ❷
```

❶ Normal positional call to the function

❷ This call has the same result despite reordering because we name the arguments.

NOTE Named arguments can't be used when calling Java functions or, conversely, when calling a Kotlin function from Java. The names of arguments aren't preserved in the bytecode to allow this. Also, altering argument names can be considered a breaking API change in Kotlin code, where in Java, only changes to type, number, or ordering of arguments will cause trouble.

Sometimes it's not necessary to pass all arguments to a function— reasonable defaults exist. In Java we accommodate this with multiple method overloads taking varying sets of arguments. Although this is supported in Kotlin too, we can use a more direct approach for default values, as shown here:

```
fun callOtherServices(value: Int, retry: Boolean = false) {
  // ...
}

callOtherServices(10)  ❶
```

❶ retry in the function is false, thanks to the default value.

Instead of needing to provide two definitions of `callOtherServices` —one with a single argument, another with the two—we can keep all the related bits in one function without boilerplate.

Another common case that Kotlin provides syntax for is single-line functions, as shown in the following code. These provide encapsulation and a name to a particular calculation or check. Kotlin supports this as part of its trend toward conciseness via an alternate declaration for such short functions:

```
fun checkCondition(n: Int) = n == 42
```

This format is shorter not only for the lack of curly braces in favor of the single `=` but also because of type inference. Return types may be excluded, and Kotlin will infer the type of the expression automatically.

This feature hints at a deeper part of Kotlin's design, which is its support for *first-class functions*. Functions in Kotlin may be passed as arguments, stored in variables and prop-

erties, and returned from other functions. Although Kotlin isn't considered a functional programming language, support for first-class functions allows Kotlin to benefit from many common patterns in functional programming.

Assigning a function to a variable can take a few different forms, depending on the source of your function. If we've previously declared the function, we can use the `::` operator to reference it by name:

```
fun checkCondition(n: Int) = n == 42
val check = ::checkCondition
```

Kotlin also has *lambda expression* syntax to create an anonymous function on the fly:

```
val anotherCheck = { n: Int -> n > 100 }
```

Regardless of how it was assigned, references to functions can be invoked as if the variable name were the function's own name. Alternatively, an `invoke` function is available, as illustrated here, if that's clearer:

```
println(check(42))              ❶
println(anotherCheck.invoke(42))  ❷
```

❶ Runs our previously fun declared checkCondition captured in the check variable and prints true

❷ Runs our lambda to see if we're greater than 100 and prints false

We're not limited to assigning functions to local variables. We can pass them to other functions as arguments like any other value. This is one of the key properties of languages with first-class functions.

As we've seen previously, Kotlin requires arguments to declare their type. Functions are no exception, and there's a specific syntax for expressing the type of functions, shown here:

```
fun callsAnother(funky: (Int) -> Unit) {      ❶
  funky(42)                                   ❷
}
```

```
callsAnother({ n: Int -> println("Got $n") })  ❸
```

❶ callsAnother takes an argument that is a function taking an Int and returning nothing.

❷ callsAnother invokes the function it is passed.

❸ We can invoke callsAnother, passing a lambda whose function type matches.

A function type is composed of two parts separated by an `->`—its list of arguments in `()` and the return type. The list of argument types can be empty, but the return type cannot be excluded. If the function you are passing doesn't return anything, its type must be specified as `Unit`.

NOTE When a lambda takes a single argument and the type can be inferred, the identifier `it` can be used, excluding the need both for a specific name and for the `->` at the start of the lambda.

Function arguments must specify the types they expect callers to pass, but Kotlin does save some typing for the caller by applying type inference to lambdas. The following are all allowed forms of our earlier invocation of `callsAnother` with progressively less and less explicit typing:

```
callsAnother({ n: Int -> println("Got $n") })  ❶
callsAnother({ n -> println("Got $n") })       ❷
callsAnother({ println("Got $it") })           ❸
```

❶ The original invocation of callsAnother with our lambda types fully specified

❷ Kotlin can infer that n must be an Int because that's what callsAnother needs.

❸ The pattern of passing a single parameter to lambda is so common that Kotlin provides special support with an implicit it parameter.

Kotlin has one more trick when passing lambdas as arguments. If a function call's last argument is a lambda, that lambda may appear after the parentheses. If the only argument to a function is a lambda, you don't even have to use parentheses at all! The following three calls are all identical:

```
callsAnother({ println("Got $it") })
callsAnother() { println("Got $it") }
callsAnother { println("Got $it") }
```

We'll dig into more details of functional programming with Kotlin in chapter 15, but let's now look at how Kotlin puts all this functional goodness to use in improving a point of frequent discontent in Java—Collections.

2.5 Collections

Collections are one of the most common data structures in programs. Java's standard Collection library, from the earliest versions, has provided a lot of power and flexibility. But constraints of both the language and backward compatibility often end up with more verbose, ceremony-laden code, especially when compared to scripting languages like Python or functional languages like Haskell. Recent Java releases have improved the situation dramatically—see chapter 1's coverage of Collections factories and appendix B for streams—but difficulties from Java's original collection design remain with us today.

Naturally, Kotlin learned from these mistakes and from day one had a seamless collection experience. Standard functions, shown next, have always existed in Kotlin for creating the most common types of Collections—a feature that arrived in Java only with version 9:

```
val readOnlyList = listOf("a", "b", "c")              ❶
val mutableList  = mutableListOf("a", "b", "c")       ❶

val readOnlyMap = mapOf("a" to 1, "b" to 2, "c" to 3)    ❷
val mutMap = mutableMapOf("a" to 1, "b" to 2, "c" to 3)  ❷

val readOnlySet = setOf(0, 1, 2)                      ❸
val mutableMap  = mutableSetOf(1, 2, 3)               ❸
```

❶ Create lists with the inferred types kotlin.collections.List<String> and kotlin.collections.MutableList<String>

❷ Create maps with the inferred types kotlin .collections.Map<String, Int> and kotlin.collections .MutableMap<String, Int>. Note the built-in syntax for defining maps with the to keyword.

❸ Create sets with the inferred types kotlin.collections.Set<String> and kotlin.collections.MutableSet<String>

The default functions return read-only copies of their Collections—a great default choice for performance and correctness. You have to explicitly ask for the `mutable` flavor to get a Collection with the interfaces for modification. Kotlin again aims to protect you from whole classes of bugs by nudging your code toward immutability as the easier, shorter option.

You may have noted that the inferred types for these Collections differ from the standard Java counterparts—though similarly named, they live in the `kotlin.collections` package. Kotlin defines its own hierarchy of collection interfaces, as shown in figure 9.1, but, under the covers, reuses the implementations from the JDK. This allows for cleaner APIs with the `kotlin.collections` interfaces, while preserving the ability to pass our Collections across to Java code, because the implementations also support the `java .util` collection interfaces.

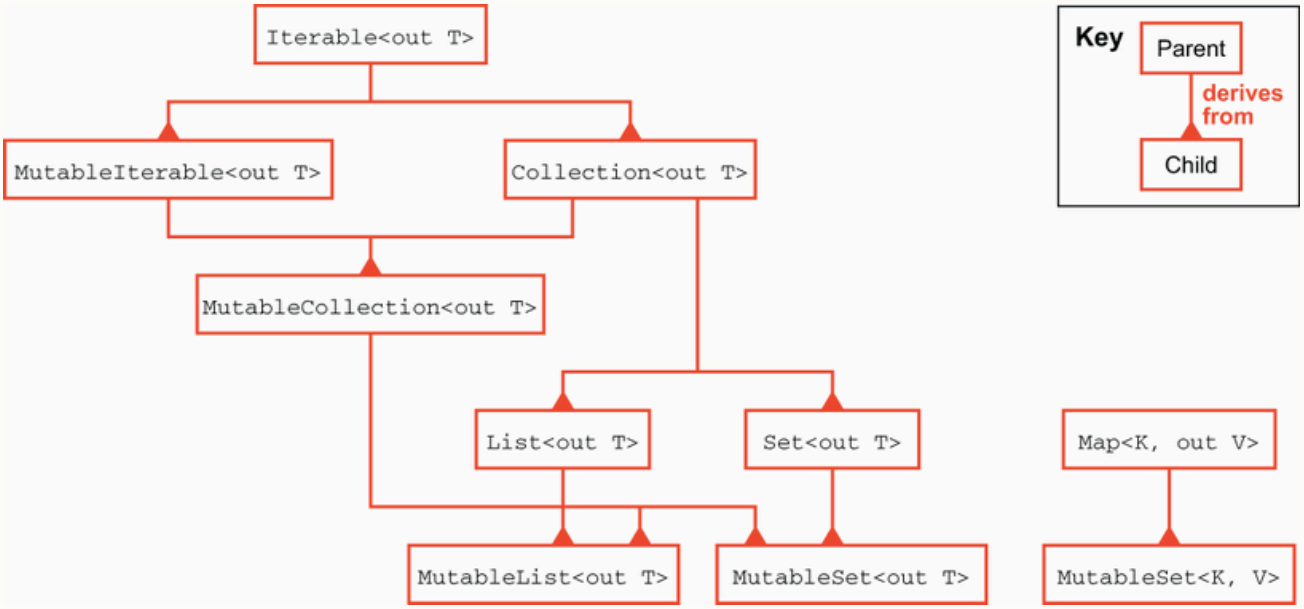


Figure 9.1 Kotlin Collection hierarchy

These Collections participate in all the standard Java interfaces and patterns. You can iterate over them with `for ... in` as follows:

```
val l = listOf("a", "b", "c")

for (s in l) {
  println(s)
}
```

However, the `for` loop to iterate over a Collection is the only operation directly in the language. A lot of other work on Collections happens over and over, and Kotlin's Collections have a ton of features using the first-class functions we saw in the previous section. These features almost always return a new Collection rather than mutating the Collection they are called against. You may have encountered this style of Collection code in Java since the release of lambdas and streams, which share many common ideas.

Often we take a value in one Collection and transform each element to a different value based on some calculation. `map` does exactly this using the function we pass in, as follows:

```
val l = listOf("a", "b", "c")
val result = l.map { it.toUpperCase() }  (1)
```

(1) result contains a list of "A", "B", "C".

Another common operation is to remove certain values from a Collection before doing further processing. `filter` expects a lambda that returns a `Boolean`. This lambda is called a *predicate*, and `filter` calls the predicate repeatedly to decide which elements to return in a new Collection, as shown next:

```
val l = listOf("a", "b", "c")
val result = l.filter { it != "b" }  (1)
```

(1) result contains a list of "a", "c".

If you care only whether a Collection fulfills certain conditions but you don't need the elements, the `all`, `any`, and `none` functions are just what you need. These avoid copying data and will return early where possible (i.e., after the first `false` for `all()`), as shown here:

```
val l = listOf("a", "b", "c")
val all  = l.all  { it.length == 1 }  (1)
val any  = l.any  { it.length == 2 }  (2)
val none = l.none { it == "a" }       (3)
```

(1) all == true

(2) any == false

❸ none == false

You can build a map from a list with the functions `associateWith` and `associateBy`. `associateWith` expects the Collection element to be the key in the resulting map. `associateBy` assumes instead the Collection element is the value in the map. If duplicates are encountered with either of these functions, the last calculated value wins, as illustrated in the next code snippet:

```
val l = listOf("!", "-", "--", "---")

val resultWith = l.associateWith { it.length }  ❶
val resultBy   = l.associateBy   { it.length }  ❷
```

❶ resultWith contains mapOf("!" to 1, "-" to 1, "--" to 2, "---" to 3).

❷ resultBy contains mapOf(1 to "-", 2 to "--", 3 to "---").

This just scratches the surface of the rich set of functions in Kotlin for working with Collections. These functions can be chained together to allow expressive, concise descriptions of operations over a Collection. The documentation is excellent with examples that walk through other topics like grouping, sorting, aggregating, and copying. See **https://kotlinlang.org/docs/collections-overview.html**.

Kotlin's focus on maintaining a tight flow between bits of code carries to other fundamental features, too. A preference for expressions over statements is another way Kotlin smooths the edges in your code.

2.6 Express yourself

Among the first structures we encounter when learning to program is `if`. In Java, `if` is a statement used to control the flow of execution through your program. Kotlin also uses `if` for this purpose, but rather than a *statement* that is just executed, `if` is an *expression* that returns a value, as shown here:

```
val iffy = if (checkCondition()) {  ❶
  "sure"                            ❷
} else {
  "nope"                            ❸
}
```

❶ The variable iffy will receive a value, depending on which branch is taken.

❷ If checkCondition() is true, “sure” will be assigned to iffy.

❸ If checkCondition() is false, “nope” will be assigned to iffy.

Like with any other variable assignment, Kotlin allows us to infer the type. In this case, the final line of each branch is accounted for in determining the type.

`if` expressions are powerful enough that Kotlin actually dropped a feature that Java inherited from C—the ternary `condition ? "sure" : "nope"` operator. The ternary shortens code, but it also has a reputation for compressing to the point of losing readability. Although it has slightly more characters, Kotlin’s version is more readable in many cases, as shown in the next code sample, and naturally converts to a multiline `if` should the logic grow further:

```
val myTurn = if (condition) "sure" else "nope"
```

In chapter 1, we discussed the introduction of *switch expressions* to Java. This is another case where Kotlin’s design preceded similar enhancements in Java. Kotlin doesn’t use the traditional C-style `switch` syntax at all but supports a powerful alternative with the keyword `when`, as follows:

```
val w = when (x) {
  1 -> "one"      ❶
  2 -> "two"      ❷
  else -> "lots"  ❸
}
```

❶ If value x is 1, then we’ll assign “one” to w.

❷ If value x is 2, then we’ll assign “two” to w.

❸ If value x is anything else, then we’ll assign “lots” to w.

`when` supports a number of other very useful forms. With the `in` keyword, you can check for membership in a Collection, as follows:

```
val valid = listOf(1, 2, 3)
val invalid = listOf(4, 5, 6)
```

```
val w = when (x) {
  in valid   -> "valid"     ❶
  in invalid -> "invalid"   ❶
  else       -> "unknown"
}
```

❶ Checks whether x is in each Collection—the equivalent of calling valid.contains(x) and invalid.contains(x)

Kotlin also has language-level support for numeric ranges, which plays nicely with `when` and `in`, too, as shown next:

```
val w = when (x) {
  in 1..3 -> "valid"      ❶
  in 4..6 -> "invalid"    ❶
  else    -> "unknown"
}
```

❶ The .. syntax defines an inclusive range, so this code is equivalent to the prior list-based example.

It's worth noting that the left-hand condition of `when` can be any valid expression as long the type matches what's required. For example, function calls can be used, a nice trick for clarifying complex conditions, as follows:

```
fun theBest() = 1
fun okValues() = listOf(2, 3)

val message = when (incoming) {
  theBest()     -> "best!"        ❶
  in okValues() -> "ok!"          ❷
  else          -> "nope"
}
```

❶ Because the return value from theBest is used directly, it must return an Int to be compared to incoming.

❷ Because the return value from okValues is used with an in, it must be a Collection.

A final point on `when`, if you aren't already convinced of its superpowers, is safety. All of our examples provide an `else` case. Removing any of those causes a compilation error complaining we haven't handled all the cases, shown here:

```
error: 'when' expression must be exhaustive, add necessary 'else' branch
```

Kotlin has one other trick for replacing a statement construct in Java with an expression—error handling with `try catch`, as shown in the following code:

```
val message = try {
  dangerousCall()          (1)
  "fine"                   (2)
} catch (e: Exception) {
  "oops"                   (3)
}
```

(1) A function call that might fail

(2) message will be assigned "fine" if we get past the dangerous call.

(3) message will be assigned "oops" if our dangerous call throws an exception.

This avoids the awkward construct of declaring a variable outside the `try catch` which all paths inside must remember to set properly. Not only is this shorter to write, but it's safer, too, because the compiler can guarantee the assignment is valid.

Although both Kotlin and Java have adopted aspects of functional programming, they are at heart object-oriented languages. Next we'll examine defining classes and objects in Kotlin.

9.3 A different view of classes and objects

Kotlin's classes provide very similar functionality to Java, starting with the keyword `class`. But, just as we've seen elsewhere, the code is different, with an emphasis on conciseness and convenience.

For starters, Kotlin doesn't use the `new` keyword for creating instances of a class. Instead, its syntax is more akin to invoking a function using the class name, as shown here:

```
val person = Person()
```

Kotlin doesn't really have fields in the same way that Java does. Instead, our friends `val` and `var` show up when declaring *properties* within the class. These can be initialized inline in the same fashion as Java fields, as follows:

```
import java.time.LocalDate

class Person {
  val birthdate = LocalDate.of(1996, 1, 23)     ❶
  var name = "Name Here"                        ❷
}
```

❶ The read-only property birthdate

❷ The mutable property name

NOTE As we saw in chapter 4, fields are present at JVM level, so Kotlin's properties are actually converted into field access in the bytecode. However, at the language level, it is better to think in terms of properties.

A major source of boilerplate in Java classes is getter and setter methods for fields. Kotlin addresses this by providing accessors for properties automatically by default. To take it a step further, Kotlin also allows us to use those accessor methods as if we were accessing a field in Java, as shown next:

```
println("Hi ${person.name}. " +                  ❶
        "You were born on ${person.birthdate}")
person.name = "Somebody Else"                    ❷
// person.birthdate = LocalDate.of(2000, 1, 1)   ❸
```

❶ Prints Hi Name here. You were born on 1996-01-23.

❷ var properties also get setters, which can be used with =.

❸ val properties cannot be set.

A big topic in class designing is visibility of state data, especially as it relates to *encapsulation*. Kotlin takes a slightly controversial move of defaulting visibility to `public`, which

differs from Java's *package-protected* default. Freely exposing all properties isn't considered good practice, but Kotlin's designers found that `public` had to be stated far more often in real code, so choosing it as the default has a major slimming effect.

Kotlin supports the following four levels of visibility, most of which align with their Java counterparts:

- *private*—Visible only within the current class or file for top-level functions
- *protected*—Visible within the class and child classes
- *internal*—Visible to the set of code you compile together
- *public*—Visible to everyone

For example, if we wanted to make `birthdate` private, it would look like this:

```
class Person {
  private val birthdate = LocalDate.of(1996, 1, 23)
  var name = "Name Here"
}
```

Because Kotlin exposes only properties, not fields, to the programmer, it opens the possibility for *delegated properties*. When followed with the `by` keyword, a property can provide a custom implementation of its getting and setting behavior. This practice will show up in a number of advanced techniques in later chapters.

Several useful delegates come with the standard library. For instance, it's not uncommon when debugging to want to know when a value was changed. `Delegates .observable` provides just such a hook, as shown here:

```
import kotlin.properties.Delegates

class Person {
  var name: String by Delegates.observable("Name Here") {
     prop, old, new -> println("Name changed from $old to $new")
  }
}
```

The value passed when calling `Delegates.observable` is treated as the initial value of the property. The lambda we pass to `Delegates.observable` will be invoked after the backing property value has been changed. A handle to the property itself, along with the old

and new values, is passed into the lambda for us to work with. Here we simply print out what changed.

Like Java, Kotlin supports constructors for creating instances of our classes, and, in fact, Kotlin actually has a few different forms for construction. The first of these is declaring a *primary constructor* at the very top with your class name, as shown next. Kotlin uses this as an alternative location where you can specify your properties as well:

```
class Person(
  val birthdate: LocalDate,                    ❶
  var name: String) {                          ❶
}

val person = Person(LocalDate.of(1996, 1, 23),  ❷
                    "Somebody Else")            ❷
```

❶ val and var in the primary constructor create properties so we don't need to declare them later.

❷ Because we didn't provide defaults, parameters must be passed at construction.

If you need visibility modifiers or annotations on a constructor, you can use a longer syntax using the `constructor` keyword. For instance, if we wanted to hide our constructor from the world, we can do it like this:

```
class Person private constructor(
  val birthdate: LocalDate,
  var name: String) {
}
```

If we want to run other logic during all object construction, Kotlin uses the `init` keyword like so:

```
class Person(
  val birthdate: LocalDate,
  var name: String) {

  init {
    if (birthdate.year < 2000) {    ❶
      println("So last century")
    }
```

```
  }
}
```

❶ init runs after we've assigned the properties from the constructor so we can access them in our code.

A class may have multiple `init` blocks, and they are run in the order in which they're defined in the class, as shown here. Properties defined in the class body are accessible to `init` blocks only after the definition:

```
class Person(
  val birthdate: LocalDate,
  var name: String) {

  init {
    // println(nameParts)                    ❶
  }

  val nameParts: List<String> = name.split(" ")

  init {
    println(nameParts)                       ❷
  }
}
```

❶ Fails to compile with error: variable 'nameParts' must be initialized

❷ Works as expected, and prints out a list

If we need additional constructors, we define them in the class body with the `constructor` keyword, as follows. These are referred to as *secondary constructors*:

```
class Person(
  val birthdate: LocalDate,
  var name: String) {

  constructor(name: String)
    : this(LocalDate.of(0, 1, 1), name) {    ❶
  }
```

❶ When a class has a primary constructor, secondary constructors must call it (directly or through other secondary constructors) via this.

NOTE Many cases in Java where multiple constructor overrides exist for supplying defaults can be handled in Kotlin with default argument values instead.

Although a class with only properties can be of use, most of the time, our classes have other functionality, too. The next code sample uses the familiar function syntax we've already seen earlier in the chapter to add a method to the class:

```
class Person(
  val birthdate: LocalDate,
  var name: String) {

  fun isBirthday(): Boolean {
    val today = LocalDateTime.now().toLocalDate()
    return today == birthdate
  }
}
```

As mentioned before, functions in Kotlin default to `public` visibility. If we want to hide a function, precede it with the desired access modifier like this:

```
class Person(
  val birthdate: LocalDate,
  var name: String) {

  fun isBirthday(): Boolean {                ❶
    return today() == birthdate
  }

  private fun today(): LocalDate {           ❷
    return LocalDateTime.now().toLocalDate()
  }
}
```

❶ isBirthday is available for anyone who can see the Person class.

❷ today is available only within the Person class.

Another key part of object-oriented programming is *inheritance*. Kotlin doesn't have the `extends` keyword but instead expresses inheritance via the familiar `:` syntax we've seen in type declarations previously and shown next:

```
class Child(birthdate: LocalDate, name: String)    ❶
  : Person(birthdate, name) {                      ❷
}
```

❶ Parameters to the Child constructor. Note these are not marked val and var, so they don't collide with the parent properties but are available as local variables to pass to the superclass constructor.

❷ Calling to the superclass constructor

Inheritance requires one other change to the `Parent` class. To encourage subclassing only where we intend and plan for it, Kotlin classes are *closed* by default. If a class may be subclassed, it must use the `open` keyword, as shown in the next code sample. This is the inverse of the situation in Java, where classes are open by default and use `final` to indicate that they may *not* be subclassed:

```
open class Person(                ❶
  val birthdate: LocalDate,
  var name: String) {
  //...
}
```

❶ open precedes the class keyword along with visibility modifiers.

The same closed-by-default principle applies to methods. The parent class must declare a method to be overridden `open`, and overrides must be marked with `override` in the child, as follows:

```
open class Person(
  val birthdate: LocalDate,
  var name: String) {

  open fun isBirthday(): Boolean {          ❶
    return today() == birthdate
  }
```

```
  private fun today() : LocalDate {
    return LocalDateTime.now().toLocalDate()
  }
}

class Child(birthdate: LocalDate, name: String)
  : Person(birthdate, name) {

  override fun isBirthday(): Boolean {          ❷
    val itsToday = super.isBirthday()           ❸
    if (itsToday) {
      println("YIPPY!!")
    }
    return itsToday
  }
}
```

❶ The Person class declares that the isBirthday function can be overridden in subclasses.

❷ The Child class must explicitly mark its method as an override.

❸ Child can call the parent implementation of the isBirthday with super.

Like Java, Kotlin allows only a single base class, but classes may inherit multiple interfaces, as shown in the next code. Kotlin's interfaces allow the default implementations of functions, much like Java has since version 8:

```
interface Greetable {
  fun greet(): String              ❶
}

open class Person constructor(
  val birthdate: LocalDate,
  var name: String): Greetable {   ❷

  override fun greet(): String {   ❸
    return "Hello there"
  }
}
```

❶ Defines our interface with a function to return a greeting

❷ Person declares that it implements Greetable.

❸ Interface functions are open, so their implementation must specify override.

In typical Kotlin style, implementing an interface uses a concise form that looks a lot like what we already use for extending a base class—no more remembering whether to use `extends` or `implements` like in Java.

3.1 Data classes

Although these basic constructs allow us to make rich object models in Kotlin, sometimes you just want a container to pass data around. Kotlin provides support for this with *data classes*.

NOTE We met Java's new `record` capability in chapter 3, and Kotlin data classes are very similar to Java records in some ways.

Kotlin already makes the property side of this seamless, but the equality issues remain with standard classes—the default `equals` and `hashCode` implementations are based on object references, rather than the values of its properties.

When we declare a type as a `data class`, though, Kotlin will create the equality functions we'd want (unless we provide an implementation ourselves explicitly), as follows:

```
class PlainPoint(val x: Int, val y: Int)

val pl1 = PlainPoint(1, 1)
val pl2 = PlainPoint(1, 1)

println(pl1 == pl2)                    ❶


data class DataPoint(val x: Int, val y: Int)

val pd1 = DataPoint(1, 1)
val pd2 = DataPoint(1, 1)

println(pd1 == pd2)                    ❷
```

❶ The default equals compares reference equality, so this prints false.

❷ With Kotlin's data class implementation, this prints true.

Data classes must have a primary constructor with a least one `val` or `var`. They can't be `open`, because it's impossible at compile time for Kotlin to correctly generate the equality functions if child classes might exist for the type. Data classes also aren't allowed as inner classes. Apart from these and a handful of more exotic constraints, though, they are normal classes on which you can implementation functions or interfaces to your heart's content.

A last feature that folks coming from Java might look for in classes is declaring a function that belongs not to instances but to the class as a whole. Kotlin chose not to support `static`, though—functions are either free-floating or are members of a type.

However, the convenience of associating functions with a class can't be denied, and Kotlin provides similar functionality via its `companion object`. This syntax declares a singleton object that lives within the class. `object` declarations in Kotlin are full objects with typical properties and functions. This avoids weird edges that `static` methods suffer from in Java (i.e., testing difficulties), while retaining the convenience of keeping functionality associated with the class.

A common use case for these functions is factory methods, where you want to keep the constructors of your objects private but allow controlled creation by more specifically named methods, as shown here:

```
class ShyObject private constructor(val name: String) {    ❶

  companion object {
    fun create(name: String): ShyObject {                  ❷
      return ShyObject(name)
    }
  }
}

ShyObject.create("The Thing")                              ❸
```

❶ ShyObject declares its constructor private so no one outside the class can use it.

❷ Our factory method inside the companion object is part of the ShyObject class, so it can access the private constructor.

❸ Outside of our class, functions on ShyObject's companion are available directly via its class name.

As a pragmatic alternative to Java, Kotlin brings a lot of convenience and boilerplate reduction to the table. But it doesn't stop there, as we'll see in the next section.

9.4 Safety

Kotlin is built on top of the JVM, and so it has no choice but to live within some design constraints that really come from the design of the virtual machine. For example, the JVM specification defines `null` as a value that can be assigned to any variable of reference type.

Despite these issues, the Kotlin language seeks to address some common code safety concerns to try to minimize the inherited pain and suffering. This manifests itself by elevating a number of Java code patterns to language features to make your code safer by default.

9.4.1 Null safety

Among the most common Java exceptions is the `NullPointerException`. This happens when we try to access a variable or field that should have contained an object but instead was `null`. Nulls have been referred to by Tony Hoare, the original creator of the Quicksort algorithm, as his "billion dollar mistake" (see **https://qconlondon.com/london-2009/qconlondon.com/london-2009/speaker/Tony+Hoare.html**), given his role in introducing the `null` reference in ALGOL.

Over its history, Java has developed several different approaches to provide protection against `null`s. The `Optional` type lets you always have a concrete object, while still indicating a "missing" value without resorting to `null`. The `@NotNull` and `@Nullable` annotations, supported by many different validation and serialization frameworks, can ensure that values aren't unexpectedly `null` at key points in our applications.

As you might expect, Kotlin has taken these common patterns and baked them right into the language itself. Let's revisit our earlier example of assigning variables. How do they behave when combined with `null`s?

```
val i: Int = null      (1)
val s: String = null   (1)
```

(1) Trying to assign null to these types will fail to compile.

Both assignments give a compilation error, `error: null can not be a value of a non-null type Int`. Although those `Int` and `String` type declarations look like Java's, they, in fact, disallow `null` values.

NOTE Kotlin has made nullability part of its type system. The Kotlin type `String` is *not* actually the same as the Java type `String` that allows `null`.

For a Kotlin variable to allow `null`, we must state it explicitly by adding a suffix of `?` to the type as follows:

```
val i: Int? = null      (1)
val s: String? = null   (1)
```

(1) Changing our types to Int? and String? will tell Kotlin to allow nulls.

NOTE Whenever possible, declare your variables and arguments with non-null types. You can rest assured that Kotlin is protecting you from those `NullPointerException` headaches.

We can't always avoid `null`s, though. Perhaps we're interacting with Java code, or our classes weren't designed with `null` safety in mind. Even once we've dipped our toes into the dangers of nullability, Kotlin still does its best to inform us of the risks, like this:

```
val s: String? = null   (1)
println(s.length)       (2)
```

(1) Creates a nullable variable

(2) Attempts to access a property on that variable

Kotlin recognizes the dereference in calling `s.length` is potentially unsafe and refuses to compile with `error: only safe (?.) or non-null asserted (!!.) calls are allowed on a nullable receiver of type String?`.

The first option Kotlin suggests is to correct this to the safe operator `?.`. This operator examines the object its being applied to. If the object is `null`, it returns `null` instead of making the further function call, like so:

```
val s: String? = null
println(s?.length)      ❶
```

❶ ?. results in printing the value null.

The safe operator returns early, so it works fine, even with a nested chain of calls. Any point along the way in our following example can safely return a `null` and the whole expression will just turn to `null`, as shown here:

```
data class Node(val parent: Node?, val value: String)    ❶

val node = getNode()                          ❷
node.parent?.parent?.parent                       ❸
```

❶ A data class that allows for an optional parent node

❷ Retrieves a Node from somewhere

❸ Sees whether node has a great-grandparent node

This may be convenient, but `?.` potentially hides data issues. If we got a `null` back on our previous great-grandparent check, we can't assure which level of our hierarchy it came from without further inspection.

The second option for our compilation failure, `( error: only safe (?.) or non-null asserted (!!.) calls are allowed on a nullable receiver of type String? .)` is using `!!` on the variable. This operator forces Kotlin to see whether the object is `null` and will raise the familiar `NullPointerException` if the value is `null`, as shown here:

```
val s: String? = null
println(s!!.length)      ❶
```

❶ Throws a NullPointerException

Although it should be needed less often, we can still check whether a variable is `null`. In fact, Kotlin can often notice such checks and let us avoid further `?.` or `!!`, like this:

```
val s: String? = null
```

```
if (s != null) {        ❶
  println(s.length)     ❷
}
```

❶ Checks for null in all cases

❷ Because we know that s isn't null, it can be safely referenced.

What we've seen here is actually a deeper feature of Kotlin called *smart casting*, which is worth looking at more closely in its own right.

4.2 Smart casting

Although good object-oriented design tries to avoid directly checking the type of objects, sometimes it's necessary. Data formats at the edges of our system may be loose about types (i.e., JSON) and often outside our control. At other times, we have plugin systems that must dynamically probe for capabilities of an object.

Kotlin embraces this need and takes it a step further in how the compiler supports the common patterns. To start, Kotlin uses the `is` operator to check an object's type as follows:

```
val s: Any = "Hello"        ❶
if (s is String) {          ❷
  println(s.toUpperCase())  ❸
}
```

❶ Any is the equivalent of Java's Object type—the base type for all objects.

❷ Checks whether s contains a String instance

❸ Uses the variable s as a String. toUpperCase wouldn't be available if the compiler treated it as type Any still within the branch.

If you're familiar with Java's `instanceof` construct, this code appears to miss a crucial step—we look to see whether `s` is a `String`, but then we don't cast it before treating it as a `String`. Fortunately, Kotlin has us covered. Within the `if` block where the compiler can ensure we have a `String`, we may use `s` as a `String` without explicitly casting. This is known as *smart casting*.

NOTE Java has a new feature called *pattern matching* that is being slowly rolled out as part of Project Amber. The first piece of it applies to `instanceof` and provides some of the same benefits as smart casting. We will discuss pattern matching in more detail in chapter 18.

Kotlin's smart casting functionality is allowed within an `if` conditional as well, as shown next:

```
val s: Any = "Hello"
if (s is String && s.toUpperCase() == "HELLO") {     ❶
  println("Got something")
}
```

❶ Kotlin can ensure the type from our check on the left-hand side of the && so it can safely uppercase without casting.

Constraints exist around where smart casting can kick in. In particular, it won't work with `var` properties on a class. This protects us against the property being mutated concurrently to a different compatible subtype after the smart cast check has been performed but before the following block executes.

Even if Kotlin can't do it directly, we may still cast to the type we expect— it's just a little less convenient, as shown here:

```
class Example(var s: Any) {
  fun checkIt() {
    if (s is String) {              ❶
      val cast = s as String        ❷
      println(cast.toUpperCase())
    }
  }
}
```

❶ Assumes s is defined in a way that we can't smart cast

❷ as casts to the expected type.

When we use `as`, we're back to the same spot as in Java when we cast. We'll see `ClassCastException` if the types aren't actually compatible. Kotlin does provide the fol-

lowing alternative if we would prefer allowing nullability into the picture instead of exceptions:

```
val cast: String? = s as? String   ❶
if (cast != null) {                ❷
  println(cast.toUpperCase())
}
```

❶ as? attempts the cast but won't throw. Note also that the resulting type is String? not String.

❷ If s couldn't be cast, the variable will be null instead.

A lot of Kotlin's power comes from taking a fresh look at the common, practical coding that Java developers have been doing for years. One area where the language provides more than just polish and protection, though, is in concurrency. Kotlin provides a technique called coroutines, which can be thought of as an alternative to the classical threading approaches that are most widely used in Java.

9.5 Concurrency

As we discussed in chapter 5, since its very first versions the JVM has supported the `Thread` class as a model of operating-system-managed threads. The thread model is well understood, but it comes with many problems.

NOTE Although threads are so deeply embedded in the Java language and ecosystem that it would be almost impossible to remove them, moving to a new, non-Java language allows us to potentially reimagine the concurrency primitives that the language might use.

Although Kotlin as a JVM language still exposes threads, it also introduces another construct called *coroutines*. At the simplest level, a coroutine can be thought of as a lighter-weight thread. These are implemented and scheduled within the runtime instead of at the operating system level, making them much less resource intensive. Spinning up thousands of coroutines isn't a problem at all, where similar counts of threads would grind a system to a halt.

NOTE We'll meet Java's take on coroutines in chapter 18 when we discuss Project Loom.

Part of Kotlin's support for coroutines is directly in the language (`suspend` functions), but to use coroutines practically requires an additional library, `kotlin-coroutine-core`. We'll see a lot more about introducing these sorts of dependencies in chapter 11, but for now the addition would look like this in Maven:

```
<dependency>
  <groupId>org.jetbrains.kotlinx</groupId>
  <artifactId>kotlinx-coroutines-core</artifactId>
  <version>1.6.0</version>
</dependency>
```

The equivalent in Kotlin-flavored Gradle follows:

```
dependencies {
  implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.6.0")
}
```

In Java a thread is started by handing it an object that implements the `Runnable` interface. Coroutines in Kotlin also need a way to receive the code to run, but instead they use the language's lambda syntax.

A coroutine is always started in a *scope*, which controls how the coroutine will be scheduled and run. We'll start with the simplest option, which is `GlobalScope`, a scope that exists for the entire duration of your application. `GlobalScope` has a `launch` function we call with a lambda to get ourselves started, as shown next:

```
import kotlinx.coroutines.GlobalScope    ❶
import kotlinx.coroutines.launch         ❶

fun main() {
  GlobalScope.launch {                   ❷
    println("Inside!")
  }
  println("Outside")                     ❸
}
```

❶ Imports for the coroutine function and object we'll use

❷ Starts a new coroutine in the GlobalScope, which lives as long as our program does

❸ Back outside of our coroutine, we'll print to see that main still runs.

When we run this example, most often you'll simply see it output the following:

```
Outside
```

Why isn't our coroutine working? We expect at some point to see `Inside` printed as well. Looking closer, though, we can spot the problem if we think about the sequence of events. `main` starts up our program. We then launch our coroutine to run asynchronously. Following that, we print our `Outside` message, and then the program is finished. When `main` is done, the program exits, regardless what coroutines may be waiting to run.

To get the result we wanted, we need to introduce a pause before the program finishes. This could be done with a loop or asking for user input at the console. We'll just use a `Thread.sleep(1000)` to give enough time for everything to settle, like this:

```
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.launch

fun main() {
    GlobalScope.launch {    ❶
        println("Inside!")
    }
    println("Outside")
    Thread.sleep(1000)      ❷
}
```

❶ Starts our coroutine again

❷ Gives the coroutine time to run

Now we'll see output with both messages, although the order is potentially nondeterministic, depending on how fast the coroutine starts up and what the main thread is doing.

At a high level, this doesn't look much different from using threads to get similar concurrent execution of code. But the underlying implementation requires fewer operating system resources (each coroutine doesn't have its own execution stack and local storage) and allows safety for operations like cancelling a coroutine.

To see this in action, we can capture a handle to the coroutine with the return value of `launch`. This coroutine object presents a `cancel` function, which we can call immediately if we want, as shown here:

```
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch

fun main() {
    val co = GlobalScope.launch {  ❶
        delay(1000)                ❷
        println("Inside!")
    }
    co.cancel()                    ❸
    println("Outside")
    Thread.sleep(2000)             ❹
}
```

❶ Captures the coroutine object returned by launch

❷ Inside of coroutines, we can call delay to wait a period of time.

❸ Cancels the coroutine immediately

❹ Wait as long as you like here—you'll never see the coroutine output.

This code will safely stop the coroutine and print only `Outside`. This is a marked contrast to the `stop()` method on `java.lang.Thread` which was deprecated long ago due to being hopelessly unsafe, as we discussed in chapter 5.

Why can coroutines accomplish this safely when threads can't? The key is the `delay` function. Its declaration is marked with a special modifier: `suspend`. Kotlin knows to treat `suspend` functions as safe spots in coroutine execution for operations like switching to another task or looking for cancellations. This is known as *cooperative multitasking*, and it's only because the code inside our coroutine "cooperates" in its calls to suspend functions that it can be cancelled.

This cooperation gives benefits beyond just the ability to safely cancel. For example, Kotlin understands when one coroutine (parent) starts another coroutine (child). Cancelling the parent automatically cancels the child coroutines without additional management on our part, as shown next:

```
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.coroutineScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch

fun main() {
    val co = GlobalScope.launch {  ❶
        coroutineScope {           ❷
            delay(1000)
            println("First")
        }
        coroutineScope {           ❸
            delay(1000)
            println("Second")
        }
    }
    co.cancel()                    ❹
    Thread.sleep(2000)             ❺
}
```

❶ Starts our parent coroutine as before

❷ Starts two child coroutines. coroutineScope associates those to the enclosing scope—in this case, our global coroutine.

❸ Starts two child coroutines. coroutineScope associates those to the enclosing scope—in this case, our global coroutine.

❹ Cancels the parent coroutine

❺ Again, we can wait here, but we won't see any output.

If you've seen the implementation necessary in Java to accomplish this sort of coordination, the value Kotlin brings here is pretty apparent.

Coroutines are a great example of how Kotlin uses its strength as a separate language with its own compiler to work with libraries to accomplish a lot of complex behavior cleanly. In fact, there's enough to coroutines that we'll be back for a deeper investigation in chapter 16.

But no language lives in a vacuum, especially on the JVM. Kotlin has seen great success and uptake because of its strong focus on interoperating with the vast world of Java

code out there.

9.6 Java interoperability

As we learned in chapter 4, the class file is the center of the JVM's execution model. It should come as no surprise that the Kotlin compiler (`kotlinc`) produces class files much like `javac` does for Java, as illustrated in figure 9.2.

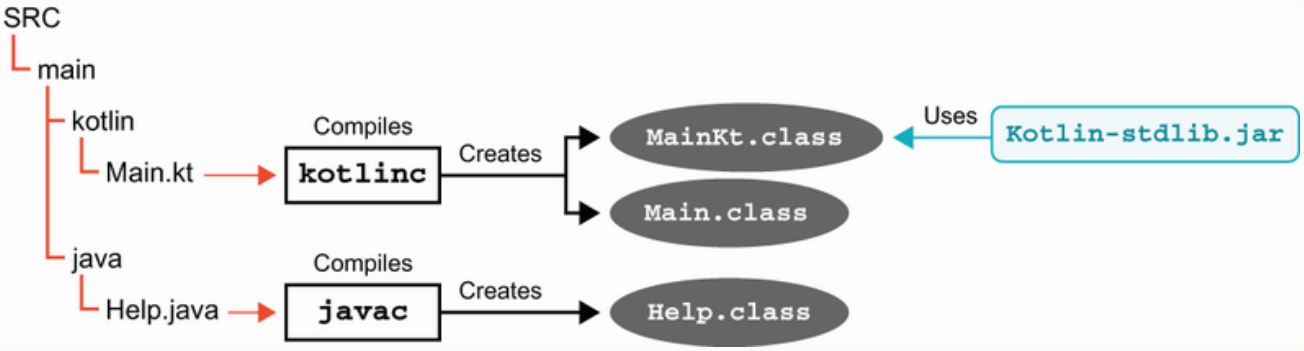


Figure 9.2 Kotlin and Java working side by side to generate class files

Basic class definitions end up looking similar between the languages, but when Kotlin provides a feature that isn't available in Java, we see more interesting differences in the generated class files. These are evidence of compiler fictions, which we discussed in chapter 8.

An example is Kotlin's top-level functions outside of classes. This isn't even directly supported in the JVM class file format. Kotlin bridges this gap by generating a class with the `Kt` suffix, named after its compilation file. Any top-level functions from within that file will appear in the `Kt` class.

NOTE You can alter the name of the generated class with the `@file:JvmName("AlternateClassName")` annotation in your `.kt` file.

For example:

```
// Main.kt                    ❶
package com.wellgrounded.kotlin    ❷

fun shout() {
  println("No classes in sight!")
}
```

❶ The filename by default influences the wrapping class name generated.

❷ Users of the function will need to import from our package as usual.

When compiled, this will produce a class file `MainKt.class` with our function in it. Because Java doesn't provide top-level functions itself, using the function from Java must go through that intermediate class as follows:

```
// Help.java
import com.wellgrounded.kotlin.MainKt;      ❶

public class Help {
  public static void main(String[] args) {
    MainKt.shout();                         ❷
  }
}
```

❶ Imports the class Kotlin created to wrap the function

❷ Invokes the function through Java's static method syntax

Another key convenience in Kotlin is its built-in treatment of properties. By using a little `val` and `var`, we never end up writing screens full of boilerplate getters and setters. Using Kotlin classes from Java reveals that at the lower level, those methods have been there all along—Kotlin just wraps them up for our convenience, as shown here:

```
// Person.kt
class Person(var name: String)              ❶

// App.java
public class App {
  public static void main(String[] args) {
    Person p = new Person("Some Body");     ❷
    System.out.println(p.getName());        ❸

    p.setName("Somebody Else");             ❹
    System.out.println(p.getName());
  }
}
```

❶ Our property is var, so it is mutable.

❷ The Kotlin class is still instantiated with new when used from Java.

❸ Accessing Person.name in Kotlin, is Person.getName() in Java.

❹ Accessing Person.name = "..." in Kotlin is Person.setName("...") in Java. Note that this accessor is available only because the Person class declares the name property a var, or mutable. If name was instead declared val, only the getName() accessor would be generated.

NOTE This example reveals that under the covers, Kotlin has been doing the standard pattern of creating a private field and wrapping access to the field. Kotlin lets us use the more natural property access form without the risks of exposing fields directly.

A number of other convenient features in Kotlin don't manifest in the resulting code when used from another JVM language. Named arguments are one example— Java just doesn't have a way to address an argument by name, so that nicety remains only in Kotlin code.

At the surface level, it might seem as though default values would suffer the same fate—after all, calling from Java requires that you explicitly pass in all arguments to the function as follows:

```
// Person.kt
class Person(var name: String) {
  fun greet(words: String = "Hi there") {     ❶
    println(words)
  }
}

// App.java
public class App {
  public static void main(String[] args) {
    Person p = new Person("Some Body");
    // p.greet();                    ❷
    p.greet("Howdy");                   ❸
  }
}
```

❶ The standard Kotlin default value for the argument words

❷ We can't invoke with default or we get a compile error, reason: actual and formal argument lists differ in length.

❸ We can pass our own value.

However, an escape hatch exists, which means we don't have to abandon Kotlin's tidiness. The `@JvmOverloads` annotation tells Kotlin to explicitly generate the necessary variations of a function, so calling it from other JVM languages looks the same:

```
// Person.kt
class Person(var name: String) {
  @JvmOverloads                    (1)
  fun greet(words: String = "Hi there") {   (1)
    println(words)
  }
}

// App.java
public class App {
  public static void main(String[] args) {
    Person p = new Person("Some Body");
    p.greet();                     (2)
    p.greet("Howdy");                (3)
  }
}
```

(1) Annotates our Kotlin function and provides the default as before

(2) Works fine and prints the default "Hi there"

(3) Works as before and prints our passed alternate greeting

Several other annotations allow for controlling how our Kotlin code manifests at the JVM level. One we've already seen in another context is `@JvmName`. This applies to functions as well as files to control the eventual naming outside Kotlin. `@JvmField` lets us avoid property wrappers and expose a bare field to the world if required.

Last, but certainly not least, is `@JvmStatic`. As we've seen earlier, Kotlin will wrap top-level functions in specially named classes, which can be accessed as static methods in Java. There's one prominent static method in all Java applications, even if you avoid statics otherwise: the `main` method that starts an application.

If we wanted to create an application in Kotlin, you can define its `main` method with `@JvmStatic` to avoid any weird naming needs on startup like this:

```
class App {                        (1)
  companion object {
```

```
    @JvmStatic fun main(args: Array<String>) {    ❷
      println("Hello from Kotlin")
    }
  }
}
```

❶ The App class we'd specify as our main class to start up.

❷ @JvmStatic means this function will present as a static method on the containing class, not just on the companion.

Making a change of language on a project is usually a huge step. Kotlin eases this burden, though, by leaning on standard patterns for multilanguage projects on the JVM. Unsurprisingly, there's additional tooling as well if you're using IntelliJ IDEA. We'll look at the standard project layout in chapter 11, but for now it's enough to know that projects typically embed the languages used into the directory layout like this:

```
.
└── src
    └── main
        ├── java
        │   └── JavaCode.java
        └── kotlin
            └── KotlinCode.kt
```

This separation makes it easy for build tools to find what they need for all your code to coexist.

If you're using IntelliJ IDEA, the good folks at JetBrains have taken this a step further. By right-clicking on a Java file, you'll find an item convert that single file directly to Kotlin. Pasting Java code into a Kotlin file will also offer this same conversion. This makes it possible to start conversion on a system from whatever point makes the most sense—perhaps with tests or a module that isn't deeply entangled in the remainder of an app.

The IDE will walk you through additional steps as necessary, but converting does take a little more than just switching some source files. Your build tools need to know about Kotlin to compile it alongside your existing code. Also, the Kotlin standard library `kotlin-stdlib` needs to be included in your project as a dependency. We'll see more about how to manage these sorts of dependencies in chapter 11.

NOTE Although IntelliJ IDEA provides a Java-to-Kotlin translation, it does not go the other direction. Its translations may also not be the ideal way to write your code in Kotlin. Keep that source control handy as always when starting a big conversion.

The fact that Kotlin compiles to class files and provides much of its additional functionality through libraries means that even including this new language in your project doesn't change that you're just running on the good old JVM.

Summary

- Kotlin is a pragmatic, attractive alternative language on the JVM.
- Kotlin draws on years of production Java use and, as a new language, made changes that Java can likely never duplicate due to backward compatibility.
- Kotlin prizes its conciseness. Familiar constructs in Java can almost always be written with less code in Kotlin.
- Safety is key for Kotlin, with `null` safety baked directly into the language to reduce `NullPointerException`s in production.
- Coroutines provide a compelling concurrency alternative to Java's classic threading model.
- Kotlin script (`kts`) enables scripting that previously would have been the domain of dynamic languages or shell.
- Even build scripts can be written using Kotlin, as we'll see in detail in chapter 11 when we discuss Gradle.

Previous chapter
< 8 Alternative JVM languages

Next chapter
10 Clojure: A different view of programming >